
WEATHER APPLICATION

Software requirements specification



Dichiaro che questo elaborato è frutto del mio personale lavoro, svolto sostanzialmente in maniera individuale e autonoma.

GIULIA AURI

Università degli studi di Modena e Reggio Emilia, Facoltà di ingegneria “Enzo Ferrari”
corso di laurea in ingegneria informatica a.a. 2024/2025

Sommario

1. introduzione	2
1.1 Obbiettivo	2
1.2 Campo d'applicazione.....	2
1.3 Fonti	2
1.4 Riferimenti.....	2
2. Descrizione generale.....	2
2.1 Macro-funzionalità del sistema.....	2
2.2 Interfaccia grafica/utente	3
2.2.1 Ricerca delle previsioni meteorologiche di una città	3
2.2.2 Autenticazione e Registrazione.....	4
2.2.3 Iscrizione alle notifiche	5
2.2.3 Modifica del profilo e disiscrizione dalle notifiche	6
2.2.4 Immagine di sfondo.....	7
2.2.5 Messaggi d'errore	8
2.2.6 Icona dell'applicazione	9
2.3 Interfaccia Hardware	9
2.4 Interfaccia Software.....	9
3. Requisiti funzionali e non funzionali del sistema	9
3.1 Requisiti funzionali	9
3.2 Requisiti non funzionali	10
3.3 Requisiti implementabili nelle prossime versioni.....	10
4. Diagrammi	10
4.1 Use case diagram	10
4.2 Class diagram	11
4.3 Sequence e Activity diagram.....	13
4.4 State diagram.....	14
5. Design pattern.....	15
5.1 Strategy pattern.....	15
5.2 Observer pattern.....	17
5.3 Singleton pattern	19

1. introduzione

La presente sezione riporta la visione globale dell'intero documento di Specifica dei Requisiti. La struttura del documento si rifà, con qualche adattamento, allo standard ISO/IEC/IEEE 29148 – 2018, noto come Software Requirements Specification.

1.1 Obiettivo

Lo scopo di questo documento è descrivere in modo chiaro e preciso la struttura, le funzionalità, i requisiti funzionali e non funzionali dell'applicazione per le previsioni meteorologiche denominata “Weather Application”.

1.2 Campo d'applicazione

L'applicazione sviluppata è “Weather Application”, ideata per visualizzare le condizioni meteorologiche, la temperatura minimo e massimo, l'umidità e la velocità del vento di una città selezionata dall'utente. Inoltre l'utente può decidere di ricevere una notifica, via mail o via sms, contenente le previsioni meteorologiche di una città da lui selezionata.

Nelle sezioni del presente documento verranno analizzate le funzionalità dell'applicazione.

1.3 Fonti

Il documento è stato redatto sulla base delle seguenti fonti, utilizzate per l'analisi dei requisiti e per l'implementazione del codice:

- Specifiche dettate da Giulia Auri, sviluppatrice del software.
- Systems and software engineering — Life cycle processes — Requirements engineering ISO/IEC/IEEE 29148 – 2018

1.4 Riferimenti

Le risorse citate all'interno del documento sono:

ISO/IEC/IEEE 29148 – 2018	https://www.iso.org/standard/72089.html
Weather forecast API: date le coordinate geografiche restituisce le previsioni meteo	https://openweathermap.org/forecast5
Geocoding API: dato il nome di una città ritorna le sue coordinate geografiche	https://openweathermap.org/api/geocoding-api

Entrambe le API sono completamente gratuite.

2. Descrizione generale

Questo software è una applicazione semplice che consente di visualizzare, sul proprio dispositivo, le previsioni meteorologiche.

2.1 Macro-funzionalità del sistema

Le principali funzionalità dell'applicazione “Weather App” sono:

- Selezionare una città e visualizzare le relative previsioni meteorologiche;

- Impostare le notifiche per una città selezionata per un determinato utente.

2.2 Interfaccia grafica/utente

2.2.1 Ricerca delle previsioni meteorologiche di una città

L'utente può digitare il nome della città per cui vuole ricercare le previsioni meteo, nel text field e, successivamente, premere il tasto invio o cliccare il tasto cerca  per attivare la selezione (rappresentato dalla lente di ingrandimento).

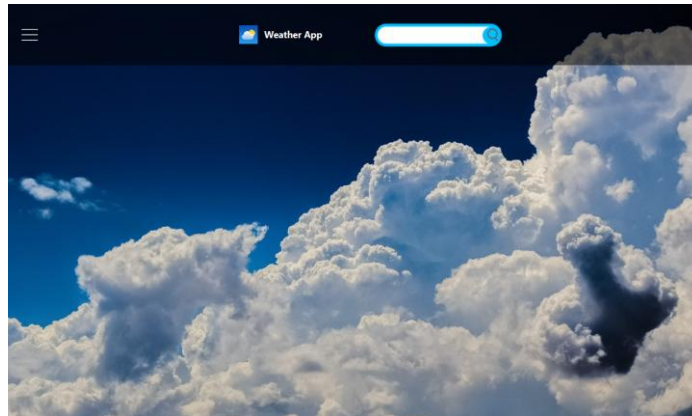


Figura 1 Schermata iniziale

Tramite una richiesta http, che verrà analizzata in seguito, in pochi secondi l'utente potrà visualizzare le condizioni meteorologiche della città selezionata e l'applicazione riporterà in output le seguenti informazioni:

- A sinistra (figura 2) le condizioni meteorologiche attuali che comprende:
 - o Temperatura minima e massima attuale
 - o Stato del meteo attuale (immagine + descrizione a parole)
 - o Percentuale di umidità attuale
 - o Velocità del vento (in km/h) attuale



Figura 2 Schermata previsioni meteo parte sinistra

- Stato del meteo
- Orario della previsione
- Temperature massime e minime

WEATHER FORECAST

Weather	Time	Temperature Min	Temperature Max
	03 mar, 21:00	7.73	10.13
	04 mar, 00:00	6.88	8.36
	04 mar, 03:00	6.07	6.07
	04 mar, 06:00	5.38	5.38
	04 mar, 09:00	10.0	10.0
	04 mar, 12:00	13.0	13.0
	04 mar, 15:00	13.69	13.69
	04 mar, 18:00	10.32	10.32
	04 mar, 21:00	8.83	8.83
			

Figura 3 Schermata previsioni meteo parte destra

2.2.2 Autenticazione e Registrazione

L'utente cliccando sull'icona  del menu a sinistra, aprirà un finestra in cui è possibile autenticarsi o registrarsi (per la prima volta), inserendo il proprio username e password.

Login

Username

Password

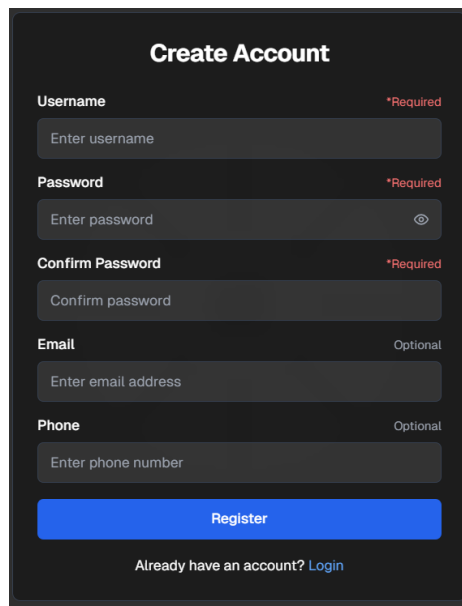


Forgot password?

Login

Don't have an account? [Register](#)

Figure 4 Login



Create Account

Username *Required

Password *Required

Confirm Password *Required

Email Optional

Phone Optional

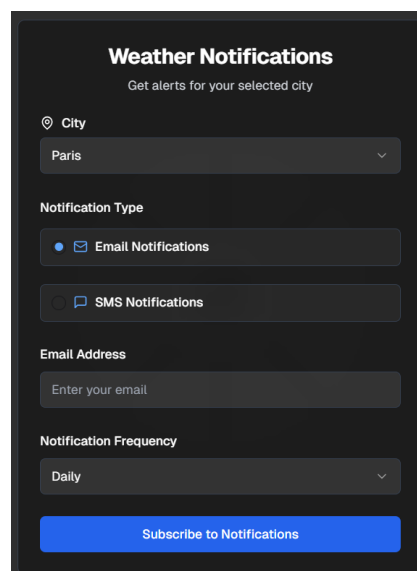
Register

Already have an account? [Login](#)

Figure 5 Registrazione dell'utente

2.2.3 Iscrizione alle notifiche

Dopo l'autenticazione o registrazione, una finestra proporrà all'utente di iscriversi alle notifiche, selezionando la città e la modalità di notifica, email o sms, che preferisce. A fianco della modalità scelta è necessario inserire l'email o il numero di telefono sul quale si desidera ricevere le previsioni meteo. Infine cliccare sul tasto "iscriviti" per rendere definitive queste modifiche. In questo modo ogni giorno l'utente riceverà una mail o un SMS contenente le previsioni meteorologiche aggiornate.



Weather Notifications
 Get alerts for your selected city

City

Notification Type

☒ Email Notifications

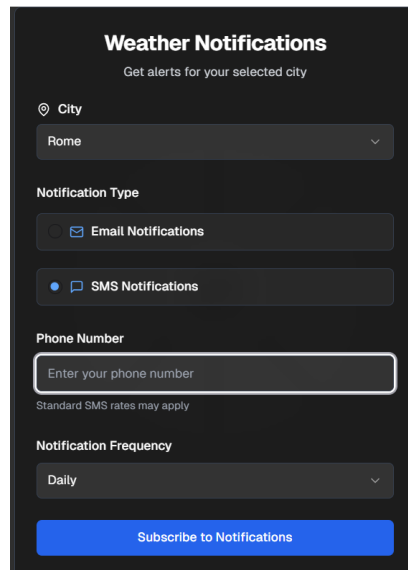
☐ SMS Notifications

Email Address

Notification Frequency

Subscribe to Notifications

Figure 6 Iscrizione alle notifiche via email



Weather Notifications
Get alerts for your selected city

City
Rome

Notification Type

☐ Email Notifications

☒ SMS Notifications

Phone Number
Enter your phone number

Standard SMS rates may apply

Notification Frequency
Daily

Subscribe to Notifications

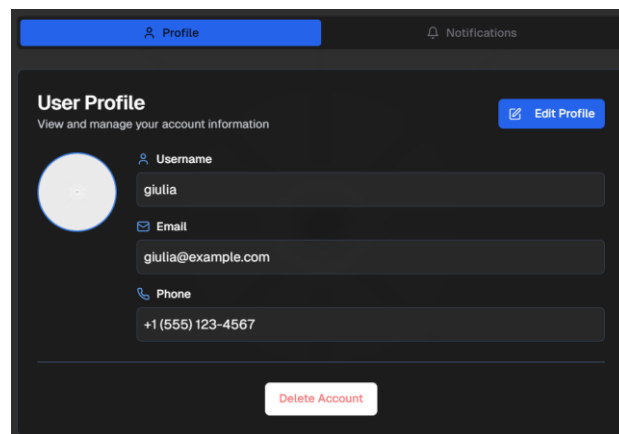
Figure 7 Iscrizione alle notifiche via SMS

2.2.3 Modifica del profilo e disiscrizione dalle notifiche

Dopo l'autenticazione o registrazione dell'utente, cliccando sul menu a sinistra ☰, comparirà la finestra come da figura 8, dalla quale l'utente potrà modificare il profilo e/o disiscrivere dal sistema di notifica di quella città.

Cliccando sul tasto “edit profile” verrà indirizzato alla schermata, simile a quella di registrazione, dove potrà modificare alcuni campi.

Cliccando “delete account” l'utente potrà annullare la sua iscrizione per una determinata città.



Profile Notifications

User Profile
View and manage your account information

Edit Profile

Username
giulia

Email
giulia@example.com

Phone
+1 (555) 123-4567

Delete Account

Figure 8 Visualizzazione dell'account

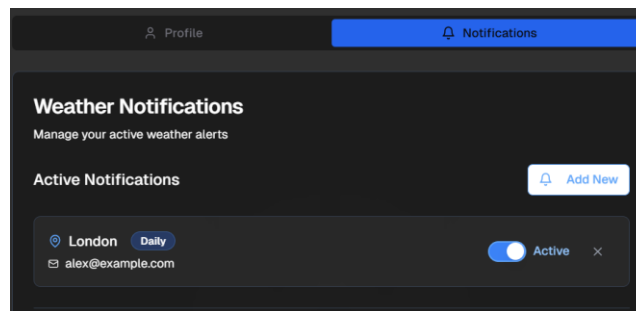


Figure 9 Visualizzazione delle notifiche attive

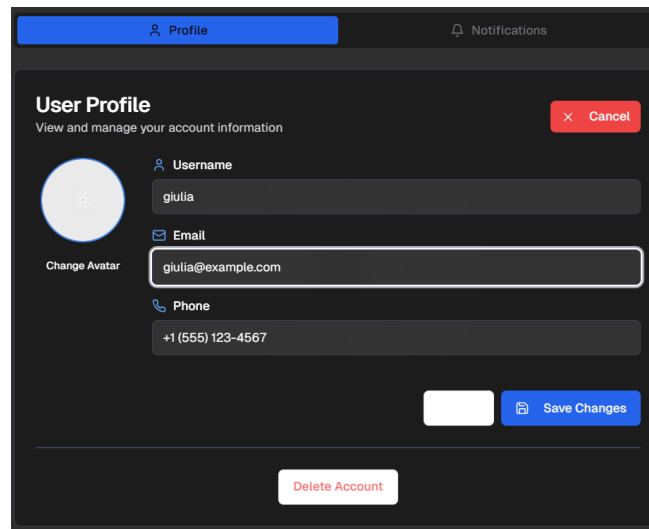


Figure 10 Modifica dell'account

2.2.4 Immagine di sfondo

L'immagine sullo sfondo viene aggiornata in base alle previsioni meteo della città che si sta cercando di visualizzare, ad esempio se è prevista pioggia verrà visualizzata un'immagine di sfondo di pioggia, oppure neve, se prevista neve, sole ... e così via.

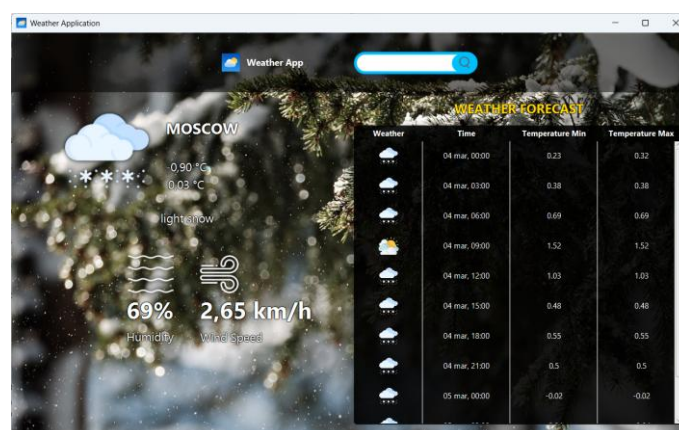


Figura 11 Schermata previsioni meteo esempio in caso di neve

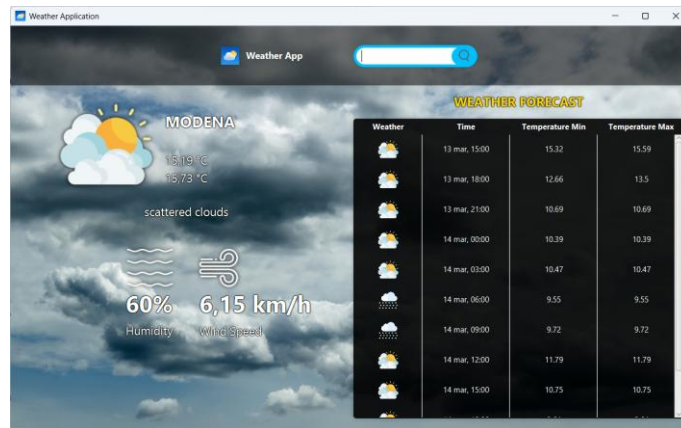


Figura 12 Schermata previsioni meteo in caso di nuvoloso

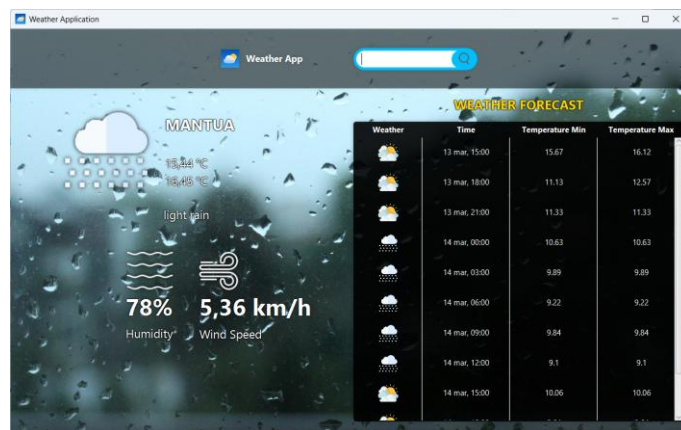


Figura 13 Schermata previsioni meteo in caso di pioggia

2.2.5 Messaggi d'errore

I messaggi d'errore permettono di segnalare all'utente quali errori si sono verificati.

Se l'utente digita città non valide ciò viene notificato all'utente tramite il seguente messaggio d'errore:



Figura 14 Errore di input

Se sono presenti errori di connessione di rete viene mostrato questo messaggio d'errore:

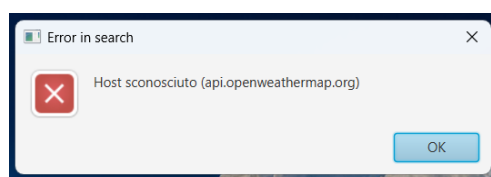


Figura 15 Errore nella richiesta al server API

2.2.6 Icona dell'applicazione

Cliccando sull'icona (figura 16) nel menu generale del dispositivo, facilmente riconoscibile, si potrà eseguire l'applicazione.

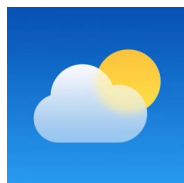


Figura 16 Icona dell'applicazione

2.3 Interfaccia Hardware

Il prodotto deve supportare una connessione internet per eseguire le richieste API.

2.4 Interfaccia Software

L'applicazione è programmata in Java ed è necessario installare, sul proprio dispositivo, una versione di Java 21.

3. Requisiti funzionali e non funzionali del sistema

La seguente sezione illustra in maniera chiara ed esaustiva i requisiti funzionali e non funzionali dell'applicazione.

3.1 Requisiti funzionali

RF01	Visualizzazione delle previsioni meteo
Descrizione	Utente digitando la città e cliccando il pulsante  o premendo il tasto invio deve poter visualizzare le previsioni meteo della città selezionata. Se la città inserita non esiste, il sistema deve mostrare un messaggio di errore.
RF02	Registrazione dell'utente
Descrizione	L'utente può registrarsi all'interno dell'applicazione inserendo i seguenti dati obbligatori: nome utente e password, e come dati facoltativi: email e numero di telefono.
RF03	Autenticazione
	Se l'utente è già registrato può accedere inserendo nome utente e password.
RF04	Abilitazione delle notifiche meteo
Descrizione	L'utente registrato può abilitare le notifiche via mail o SMS, inserendo la mail o il numero di telefono su cui riceverle.
RF05	Modifica del profilo
Descrizione	L'utente già registrato può modificare i dati personali tra cui: email, numero di telefono e password.
RF06	Disabilitazione delle notifiche meteo
Descrizione	L'utente registrato può eliminare il servizio di ricezione di notifiche per una città
RF07	Invio delle notifiche
Descrizione	Il sistema invia automaticamente le notifiche agli utenti registrati nelle modalità scelte: via email o SMS

3.2 Requisiti non funzionali

RNF01	Performance
Descrizione	L'applicazione restituisce le previsioni meteo in tempi rapidi, garantendo un'esperienza fluida. Se il sistema non riceve una risposta alla richiesta entro 1 sec, un'interfaccia mostra all'utente che si è verificato un errore.
RNF02	Usabilità
Descrizione	L'applicazione è semplice ed intuitiva e può essere utilizzata da qualsiasi utente.
RNF03	Manutenibilità
Descrizione	Il software dovrà essere mantenibile in modo da poter rilasciare una nuova versione, con funzionalità potenziate.
RNF04	Portabilità
Descrizione	Per poter utilizzare questa applicazione meteo è necessario dotare il proprio dispositivo di linguaggio Java.
RNF05	Affidabilità
Descrizione	L'applicazione riporta previsione meteorologiche affidabili basate su API openweathermap.
RNF06	Scalabilità
Descrizione	L'applicazione può gestire un numero elevato di utenti senza subire rallentamenti.

3.3 Requisiti implementabili nelle prossime versioni

Requisiti da analizzare/realizzare in futuro, nella versione 2.0:

- Permettere all'utente di selezionare ulteriori tipologie di notifiche (oltre all'email e al sms già implementati) come ad esempio telegram, notifica push, ...
- Aggiungere tasti per ulteriori personalizzazioni dell'interfaccia come, ad esempio l'unità di misura della temperatura: Fahrenheit o Celsius.
- Permettere all'utente di poter scegliere più città preferite per cui ricevere notifiche.
- Permettere all'utente la frequenza con cui ricevere le previsioni meteo.
- Implementare l'invio di allerte meteo per le città di cui si è scelto di ricevere le notifiche.

4. Diagrammi

Questa sezione illustra, tramite diagrammi UML, l'implementazione e le funzionalità dell'applicazione.

4.1 Use case diagram

Il seguente diagramma UML, figura 17, ha lo scopo di schematizzare i requisiti funzionali dell'applicazione, in particolare le funzionalità dell'utente e quelle gestite dal sistema.

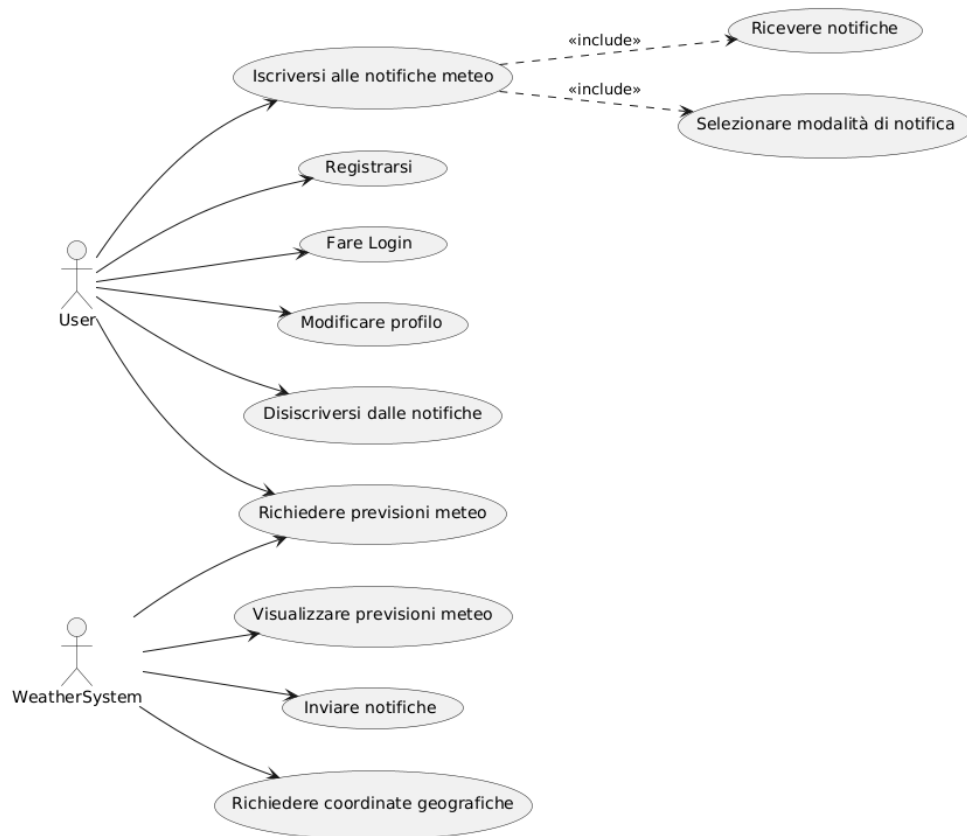


Figura 4 Use case diagram

4.2 Class diagram

Il seguente class diagram, figura 18, ha lo scopo di mostrare le classi, con metodi e attributi, e le relazioni tra di esse.

- Nel class diagram sono presenti interfacce e classi astratte come, rispettivamente, “Observer” e “RequestMarker” che definiscono le funzionalità di base supportando il riutilizzo del codice.
 - Le “EmailObserver” e in “SMSObserver” implementano la classe Observer, implementando 2 differenti tipologie di notifiche.
 - Rispettando il dependency inversion principle le classi “WeatherRequest” e “GeocoordinateRequest” dipendono da “RequestMarker”.
- L'applicazione gestisce la richiesta API di una specifica città e un sistema di notifiche che può essere via sms (“SMSObserver”) e via email (“EmailObserver”), garantendo la registrazione degli utenti (“UserRegistry”). Inoltre prevede la visualizzazione delle previsioni tramite interfaccia grafica (“Controller”).
- Ogni classe rispetta il single responsibility principle occupandosi di una singola funzionalità, per esempio “UserRegistry” si occupa della registrazione degli utenti, “GeoCoordinateRequest” esegue la richiesta API delle coordinate geografiche corrispondenti alla città inserita.
- L'interfaccia “Observer” può essere estesa facilmente implementando una nuova modalità di notifica rispettando il principio dell'Open-closed.
- La classe “RequestMarker” può essere sostituita a “WeatherRequest” seguendo il “Liskov Substitution Principle”.

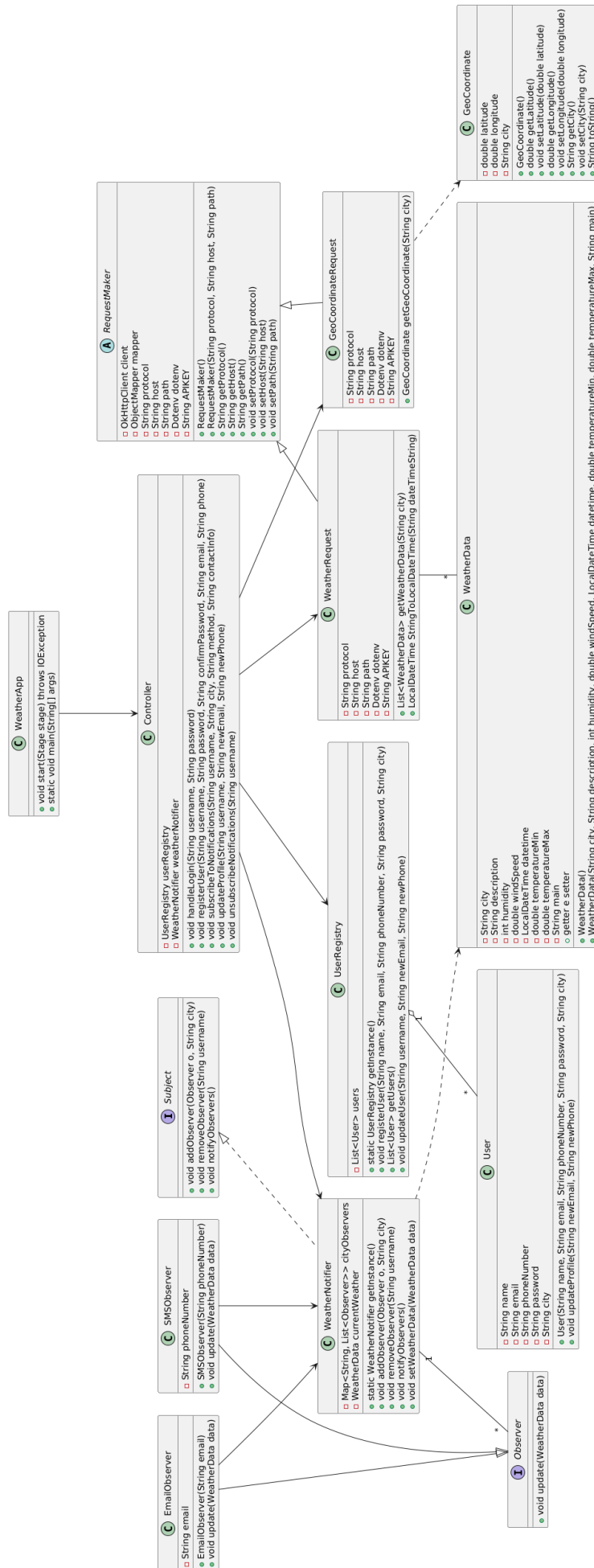


Figure 18 Class diagram

4.3 Sequence e Activity diagram

I seguenti sequence diagram e activity diagram, rispettivamente figura 19 e figura 20, spiegano ciò che accade quando l'utente clicca sul bottone 📍 dopo aver digitato il nome della città per cui si vogliono ricercare le previsioni meteo.

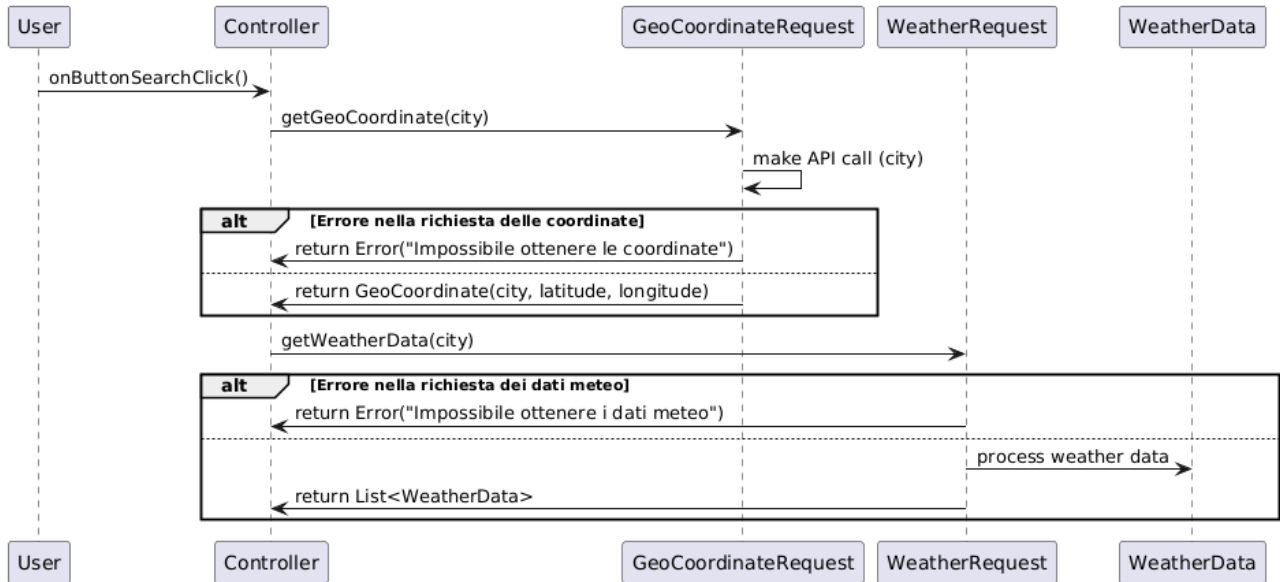


Figura 19 Sequence diagram

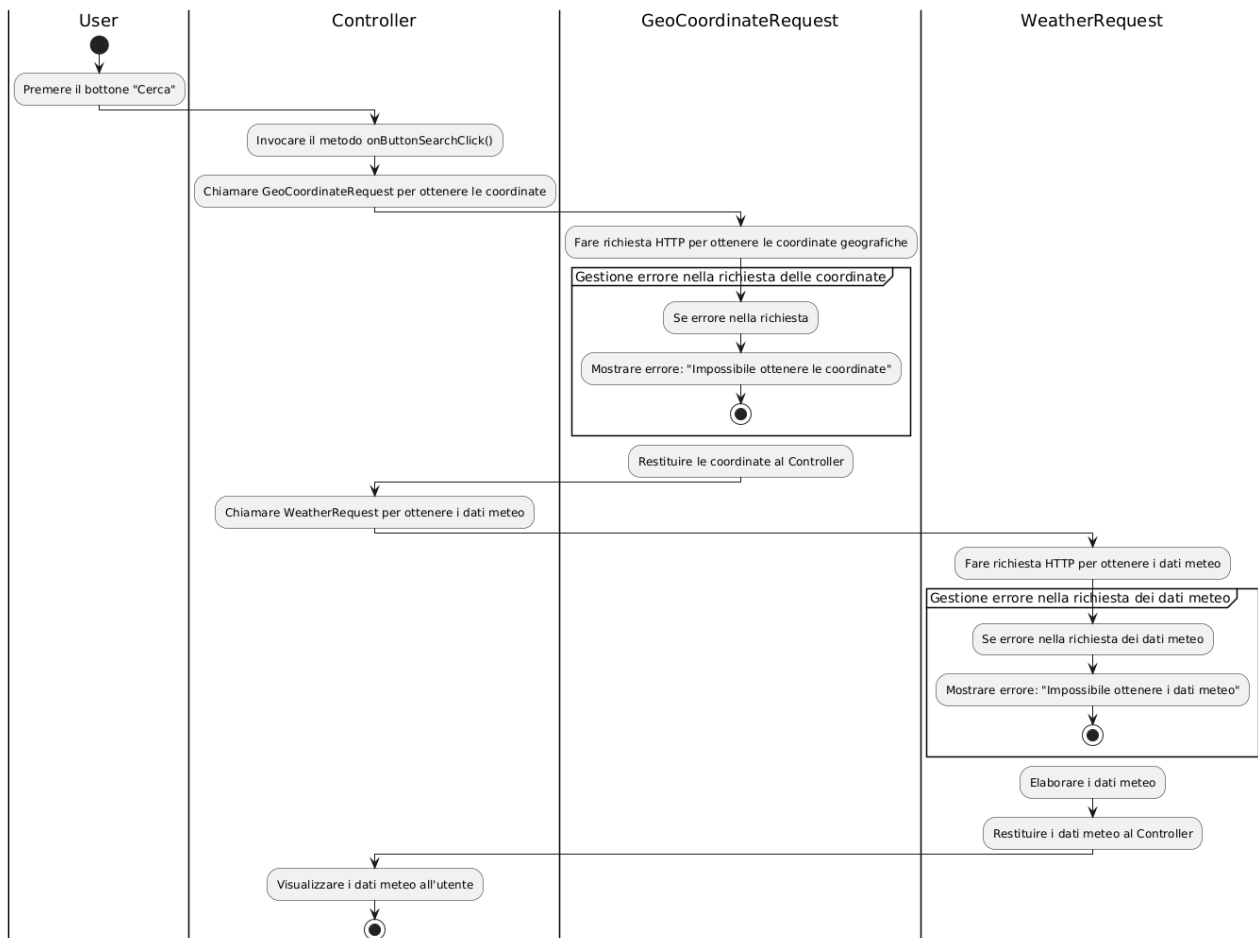


Figura 20 Activity diagram

4.4 State diagram

Il seguente state diagram, figura 21, mostra il comportamento dinamico assunto dal sistema quando viene effettuata una richiesta http all'API. La richiesta viene creata quando l'utente clicca il bottone



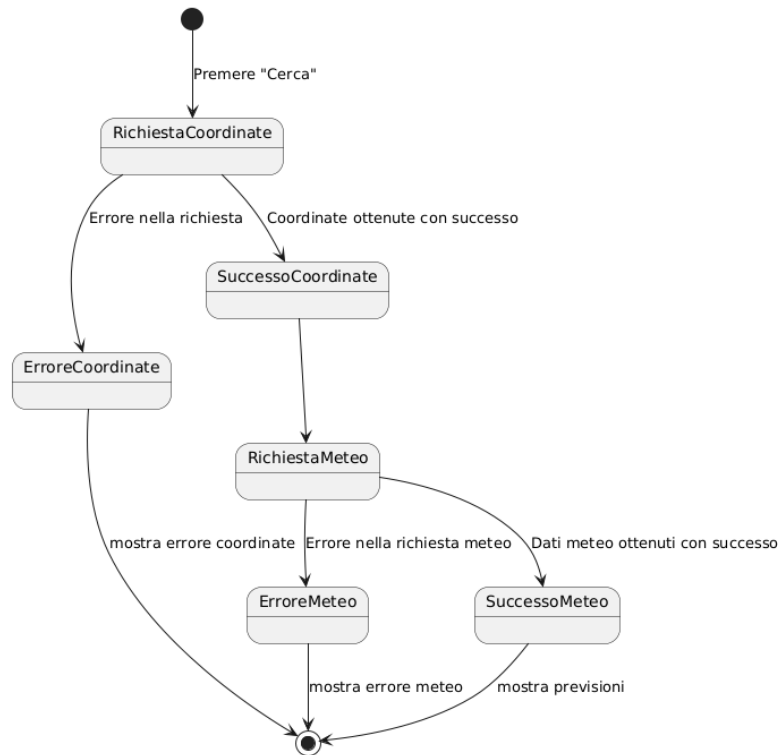


Figura 21 State diagram

5. Design pattern

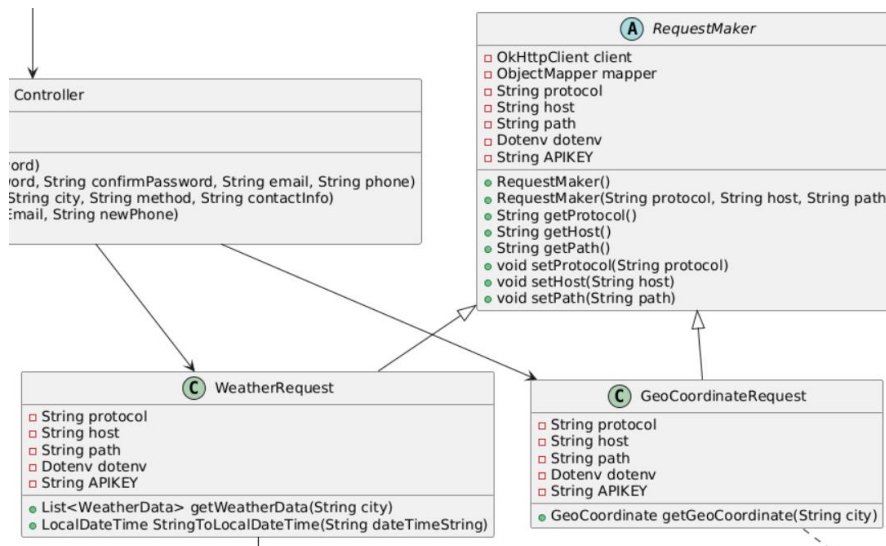
Questa sezione ha lo scopo di illustrare i design pattern utilizzati all'interno della struttura del software della “Weather App”, rispettando i principi fondamentali dell'ingegneria del software

5.1 Strategy pattern

Si è scelto di adottare questo pattern per incapsulare algoritmi simili evitando la ripetizione del codice. La struttura del codice di richiesta API, per la restituzione delle coordinate geografiche, è molto simile alla richiesta API delle previsioni meteo. Permettendo in questo modo di dividere gli elementi che cambiano da quelli che rimangono stabili e di selezionare l'algoritmo appropriato a runtime. Le differenze tra le richieste riguardano principalmente due aspetti:

1. L'URL a cui viene fatta la richiesta: nel caso delle coordinate geografiche, viene utilizzato l'endpoint per le coordinate, mentre per le previsioni meteo viene usato un endpoint diverso.
2. Il tipo di dati restituiti: nel caso delle coordinate, i dati restituiti riguardano la latitudine, la longitudine e la città (riassunti nell'oggetto “GeoCoordinate”), mentre nel caso delle previsioni meteo viene restituita una lista di dati meteorologici (oggetti di tipo “WeatherData”).

Questo pattern rende flessibile la nostra struttura; se nella prossima versione si vorrà implementare un'altra funzionalità, che necessita di richiesta API, sarà possibile farlo estendendo la classe astratta “RequestMarker” senza modificare il codice esistente.



```

public abstract class RequestMarker<E> {
    protected OkHttpClient client;
    protected ObjectMapper mapper;

    protected String protocol;
    protected String host;
    protected String path;
    protected Dotenv dotenv;
    protected String APIKEY;

    public RequestMarker() {
    }

    public RequestMarker(String protocol, String host, String path) {
        this.client = new OkHttpClient();
        this.mapper = new ObjectMapper();
        this.protocol = protocol;
        this.host = host;
        this.path = path;
        this.dotenv = Dotenv.configure()
            .directory("../.env")
            .load();
        this.APIKEY = dotenv.get("API_KEY");
    }

    abstract List<E> executeRequest(String city);
}

public class GeoCoordinateRequest extends RequestMarker<GeoCoordinate>{

    public GeoCoordinateRequest() {
        super( "https","api.openweathermap.org","geo/1.0/direct");
    }

    public List<GeoCoordinate> executeRequest(String city) {

        // implementazione della richiesta HTTP
    }

}

```

```

public class WeatherRequest extends RequestMarker<WeatherData>{

    public WeatherRequest(GeoCoordinate geocoordinate) {
        super("http","api.openweathermap.org", "data/2.5/forecast");
        this.geocoordinate = geocoordinate;
    }

    GeoCoordinate geocoordinate;

    public List<WeatherData> executeRequest(String city){

        //implementazione della richiesta HTTP

    }

}

```

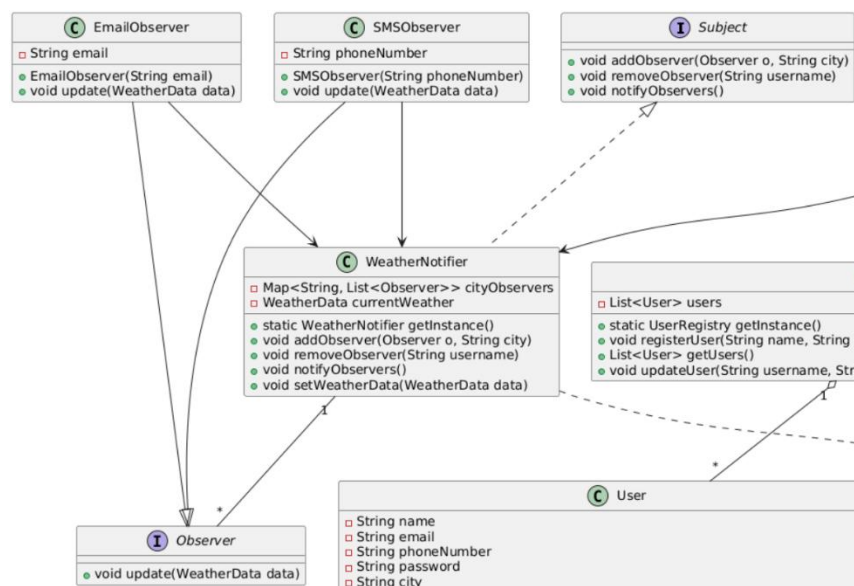
5.2 Observer pattern

È stato deciso di utilizzare il seguente pattern per permettere di notificare, via mail o SMS, a tutti gli utenti registrati, come Observer in modo automatico, le previsioni meteo per la città che era stata indicata al momento della registrazione, senza limitazioni di numero di unità registrate.

In questa applicazione, la classe “WeatherNotifier” (implementazione concreta di “Subject”) è utilizzata per notificare a tutti gli utenti (osservatori) registrati ogni giorno quando i dati meteo vengono aggiornati. La classe “WeatherNotifier” contiene una variabile di tipo mappa, chiamata “cityObservers” che, per ogni città, memorizza una lista di utenti a cui mandare una notifica contenente le previsioni meteo di quella città.

Questo pattern permette ad ogni utente di iscriversi e di ricevere notifiche solo per la città di interesse. Se si aggiungono nuovi utenti, non è necessario modificare il codice per le altre città, di conseguenza rende l’applicazione scalabile.

L’Observer pattern rende l’applicazione flessibile qualora, in futuro, si volessero aggiungere altri tipi di notifiche come Telegram, push notification, semplicemente si potrebbe realizzare creando nuove classi che implementano l’interfaccia “Observer”, senza modificare il codice esistente.



```

public interface Observer {
    void update(WeatherData data);
}

public interface Subject {
    void addObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}

public class WeatherNotifier implements Subject {
    private static WeatherNotifier instance;
    private final Map<String, List<Observer>> cityObservers;

    private WeatherNotifier() {
        cityObservers = new HashMap<>();
    }

    public static WeatherNotifier getInstance() {
        if (instance == null) {
            instance = new WeatherNotifier();
        }
        return instance;
    }

    @Override
    public void addObserver(Observer observer, String city) {
        cityObservers.computeIfAbsent(city, k -> new
ArrayList<>()).add(observer);
    }

    @Override
    public void removeObserver(Observer observer, String city) {
        List<Observer> observers = cityObservers.get(city);
        if (observers != null) {
            observers.remove(observer);
        }
    }

    @Override
    public void notifyObservers(WeatherData data) {
        List<Observer> observers = cityObservers.get(data.getCity());
        if (observers != null) {
            for (Observer observer : observers) {
                observer.update(data);
            }
        }
    }
}

public class EmailObserver implements Observer {
    private String email;
    private String city;

    public EmailObserver(String email, String city) {
        this.email = email;
        this.city = city;
    }

    @Override
    public void update(WeatherData data) {
        //implementazione invio previsioni via email
    }

    public String getCity() {

```

```

        return city;
    }
}

public class SMSObserver implements Observer {
    private String phoneNumber;

    public SMSObserver(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

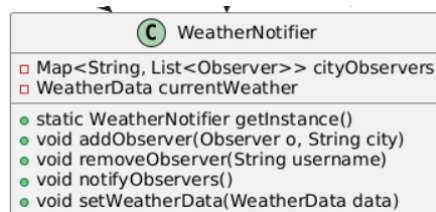
    @Override
    public void update(WeatherData data) {
        //implementazione invio previsioni via SMS
    }
}

```

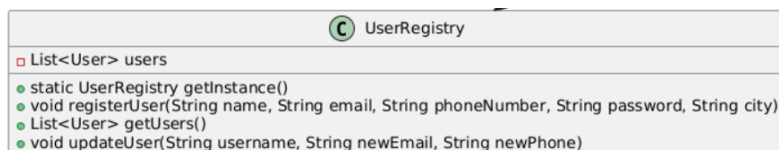
5.3 Singleton pattern

Questo pattern assicura che esista una sola istanza, per ciascuna classe, durante l'esecuzione del programma, garantendo un punto di accesso globale ed il controllo centralizzato dello stato.

La classe “WeatherNotifier” è responsabile della gestione delle notifiche meteo per gli utenti registrati e il sistema dovrà avere un'unica istanza che gestisca tutti gli observer e notifichi le previsioni meteo. Per questo motivo si è deciso di adottare il design pattern singleton alla classe “WeatherNotifier”.



Anche alla classe “UserRegister”, responsabile della gestione degli utenti registrati nel sistema, si può applicare il pattern singleton per garantire che esista un'unica lista di utenti registrati ed evitare che esistano più istanze di questa classe. Utilizzando questo pattern il sistema gestirà un accesso controllato alle classi “UserRegister” e “WeatherNotifier”.



```

public class WeatherNotifier implements Subject {
    private static WeatherNotifier instance;
    private final Map<String, List<Observer>> cityObservers;

    private WeatherNotifier() {
        cityObservers = new HashMap<>();
    }

    public static WeatherNotifier getInstance() {
        if (instance == null) {
            instance = new WeatherNotifier();
        }
        return instance;
    }
}

```

```

@Override
public void addObserver(Observer observer, String city) {
    cityObservers.computeIfAbsent(city, k -> new
ArrayList<>()).add(observer);
}

@Override
public void removeObserver(Observer observer, String city) {
    List<Observer> observers = cityObservers.get(city);
    if (observers != null) {
        observers.remove(observer);
    }
}

@Override
public void notifyObservers(WeatherData data) {
    List<Observer> observers = cityObservers.get(data.getCity());
    if (observers != null) {
        for (Observer observer : observers) {
            observer.update(data);
        }
    }
}
}

public class UserRegistry {
    private static UserRegistry instance;
    private List<User> users;

    private UserRegistry() {
        users = new ArrayList<>();
    }

    public static UserRegistry getInstance() {
        if (instance == null) {
            instance = new UserRegistry();
        }
        return instance;
    }

    public void registerUser(String name, String email, String password,
String city) {
        users.add(new User(name, email, password, city));
    }

    public List<User> getUsers() {
        return users;
    }
}

```